



École Nationale de la Statistique
et de l'Analyse de l'Information

Première année — Projet Traitement de Données

Application de Consultation de Statistiques Sportives

Membres du groupe

Maxime Bounea
Martin Dubos
Nolann Bougis
Zoeb Gouloumaly

Tuteur pédagogique

Clément Valot

Année universitaire 2025–2026

Table des matières

Introduction	1
1 Présentation du projet	2
1.1 Contexte	2
1.2 Besoins fonctionnels	2
1.3 Présentation des données	3
2 Conception de l'application	4
2.1 Modélisation	4
2.2 Architecture générale	4
2.3 Choix technologiques	5
3 Réalisation	6
3.1 Organisation du travail	6
3.2 Développement	6
3.3 Difficultés rencontrées	7
4 Tests et validation	8
4.1 Stratégie de test	8
4.2 Résultats des tests	8
4.3 Limites de l'application	8
5 Conclusion	10

Introduction

Dans le cadre du cours de Traitement de Données de première année à l'ENSAI, nous avons développé une application Python de consultation et de visualisation de statistiques sportives. Ce projet nous a permis de mettre en pratique les concepts de programmation orientée objet, de manipulation de données et de travail collaboratif via Git.

L'omniprésence des données dans le monde du sport moderne soulève une problématique centrale : comment concevoir une application modulaire capable d'agréger, de traiter et de restituer des données sportives hétérogènes de manière claire et accessible ?

L'objectif de notre application est de permettre à tout utilisateur de consulter facilement des statistiques sportives : rechercher un joueur ou une équipe, comparer deux entités, visualiser l'évolution d'un classement sur plusieurs saisons, ou encore explorer les résultats d'une compétition.

Ce rapport s'organise de la manière suivante : le chapitre 1 présente le projet et les données utilisées, le chapitre 2 décrit la conception et l'architecture, le chapitre 3 détaille la réalisation et les choix techniques, le chapitre 4 présente les tests et leurs résultats, et le chapitre 5 conclut avec un bilan et des perspectives d'amélioration.

Chapitre 1

Présentation du projet

1.1 Contexte

[À COMPLÉTER — reformuler le cahier des charges fourni par l'ENSAI]

Dans le cadre de ce projet, nous avons choisi de développer une application de consultation de statistiques multi-sports. Nous avons interprété le sujet en privilégiant la diversité des sports couverts et la richesse des fonctionnalités offertes à l'utilisateur. Notre application couvre cinq sports : le tennis (circuits ATP et WTA), le football (ligues européennes), le basketball (NBA), le badminton et le volleyball (Jeux Olympiques de Paris 2024).

Nous avons opté pour une interface en ligne de commande (CLI), conforme aux contraintes du projet, et une architecture orientée objet permettant d'ajouter facilement de nouveaux sports sans modifier le cœur du code.

1.2 Besoins fonctionnels

L'application répond à huit grandes fonctionnalités :

F1 — Chargement des données : L'utilisateur peut charger les données CSV d'un ou plusieurs sports. Les données sont ensuite sérialisées via pickle afin d'éviter de relire les fichiers à chaque lancement, ce qui réduit considérablement les temps de démarrage.

F2 — Recherche d'un joueur : L'utilisateur peut rechercher un joueur par son nom (ou une partie de son nom). La recherche est insensible à la casse. En cas d'homonymes, l'application propose une liste et invite l'utilisateur à choisir. La fiche du joueur affiche ses statistiques globales calculées depuis les matchs (taux de victoire, aces, double-fautes pour le tennis, etc.).

F3 — Statistiques d'une équipe : Pour les sports collectifs (football, basketball), l'utilisateur peut consulter les statistiques d'une équipe pour une saison donnée : classement, points, buts marqués et encaissés pour le football ; PPG, RPG, APG, Net Rating, eFG% pour le basketball.

F4 — Confrontations directes (H2H) : L'application affiche l'historique des confrontations entre deux joueurs ou deux équipes, avec le nombre de victoires de chaque côté et les éventuels matchs nuls.

F5 — Vue détaillée par compétition : Pour le football, l'utilisateur peut afficher le classement complet d'un championnat pour une saison donnée. Pour le basketball, une vue

détaillée de la NBA présente les conférences Est et Ouest, le play-in et les playoffs.

F6 — Évolution graphique : L'application génère un graphique matplotlib montrant l'évolution du classement ou des points d'un joueur ou d'une équipe saison par saison.

F7 — Module Volleyball JO 2024 : Un module dédié permet de consulter les résultats de la compétition de volleyball aux Jeux Olympiques de Paris 2024, les statistiques par équipe (ratio de sets, profil des victoires), les profils des athlètes (taille par pays, distribution des âges) et l'encadrement technique.

F8 — Ajout de jeux de données : L'utilisateur peut ajouter un nouveau jeu de données CSV via le menu, sans modifier le code source.

Les contraintes techniques imposées sont les suivantes : langage Python 3, paradigme orienté objet, gestion de versions avec Git et GitHub, tests unitaires, données en entrée sous forme de fichiers CSV uniquement, interface en ligne de commande.

1.3 Présentation des données

Notre application exploite cinq jeux de données distincts :

Tennis : Les données proviennent du dépôt public de Jeff Sackmann (`tennis_atp` / `tennis_wta`). Elles contiennent les résultats de la saison 2024, avec 443 joueurs ATP, plusieurs centaines de joueuses WTA, et plusieurs milliers de matchs. Chaque match contient des statistiques détaillées de service (aces, double-fautes, premier service, balles de break).

Football : Les données sont issues de la European Soccer Database (Kaggle), couvrant les principales ligues européennes sur plusieurs saisons. La base contient plusieurs milliers de joueurs, d'équipes et de matchs, répartis sur plusieurs ligues (Premier League, La Liga, Bundesliga, Serie A, Ligue 1...).

Basketball : Les données NBA proviennent de Kaggle et couvrent plusieurs saisons. Elles incluent les résultats des matchs de saison régulière et des playoffs, ainsi que des statistiques avancées par équipe (Net Rating, eFG%, Pace...).

Badminton : Un jeu de données contenant les joueurs et les résultats de matchs internationaux.

Volleyball : Les données des Jeux Olympiques de Paris 2024 (tournois masculin et féminin) incluent les matchs, les profils des joueurs et joueuses, ainsi que les staffs techniques de chaque équipe nationale.

Ces jeux de données ont été choisis pour leur richesse, leur disponibilité en accès libre et la diversité des sports qu'ils représentent. Chaque jeu de données a nécessité un traitement spécifique en raison de formats différents (noms de colonnes, encodages, structures des saisons).

Chapitre 2

Conception de l'application

2.1 Modélisation

Notre modèle objet s'articule autour de quatre classes principales.

La classe **Player** représente un joueur, quelle que soit la discipline. Elle stocke les informations communes : identifiant, nom, prénom, date de naissance, pays, taille, poids, position et genre. Elle possède un dictionnaire **statistiques** qui associe à chaque saison un ensemble de statistiques calculées.

La classe **Team** représente une équipe ou un pays selon le sport. Elle contient le nom, le nom court et un dictionnaire de statistiques par saison.

La classe **Match** est la classe de base pour tous les matchs. Elle stocke les identifiants des deux entités qui s'affrontent, les scores, la date et la saison. Elle expose une méthode **vainqueur_id()** qui retourne l'identifiant du vainqueur (ou **None** en cas de match nul). Quatre sous-classes héritent de **Match** en ajoutant les attributs spécifiques à chaque sport :

- **MatchTennis** : surface, tour, tournoi, circuit (ATP/WTa), statistiques de service du vainqueur et du perdant
- **MatchFootball** : identifiant de la ligue, compositions des équipes domicile et extérieur
- **MatchBasketball** : type de saison (saison régulière, playoffs, play-in)
- **MatchVolley** : genre (H/F) et phase de la compétition

La classe **Sport** est l'agrégateur central. Elle contient les dictionnaires de joueurs et d'équipes (indexés par identifiant) ainsi que la liste des matchs. Elle expose des méthodes de recherche (**get_joueur_matches**, **get_equipe_matches**) et de calcul du classement (**calculer_classement**).

[À COMPLÉTER — insérer ici le diagramme de classes UML et le diagramme de cas d'utilisation]

2.2 Architecture générale

L'application est organisée en quatre couches :

La **couche Interface** (**__main__.py**) gère l'interaction avec l'utilisateur : affichage des menus, saisie des entrées, appel aux fonctions d'affichage et de visualisation. C'est le point d'entrée unique de l'application.

La **couche Analyse** (`src/Analysis/`) regroupe les modules d'analyse avancée, notamment le module `GoatFinder` qui permet de comparer les performances des meilleurs joueurs.

La **couche Chargement** (`src/Parsers/`) est responsable de la lecture et du parsing des fichiers CSV. Elle est organisée en trois familles de loaders : `MatchLoader`, `PlayerLoader` et `TeamLoader`. Chaque famille dispose d'un dispatcher principal qui délègue au loader spécifique au sport (ex. `TennisMenPlayerLoader`, `FootballPlayerLoader...`).

La **couche Modèle** (`src/Model/`) contient les classes métier : `Player`, `Team`, `Match` et ses sous-classes, `Sport`.

Le flux de données est le suivant : les fichiers CSV sont lus par les Parsers qui créent des objets du Modèle, lesquels sont regroupés dans un objet `Sport`. Cet objet est ensuite sérialisé via pickle pour être rechargé rapidement lors des sessions suivantes. Lors de la consultation, `__main__.py` charge l'objet depuis le pickle et l'interroge.

En termes de métriques, le projet compte environ [À COMPLÉTER] lignes de code réparties sur [À COMPLÉTER] fichiers Python.

2.3 Choix technologiques

Python 3 est le langage imposé par le cahier des charges. Il offre un large écosystème de bibliothèques pour le traitement de données.

pandas a été choisi pour la lecture et la manipulation des fichiers CSV. Il gère nativement les valeurs manquantes (NaN), facilite les agrégations et les filtres, et est le standard de facto pour la data en Python.

matplotlib a été retenu pour la visualisation. Il permet de générer des graphiques en barres, des courbes d'évolution, des histogrammes et des camemberts. Son mode non-interactif (**Agg**) permet également de sauvegarder les figures en PNG.

pickle est utilisé pour la sérialisation des objets Python. Il permet de mettre en cache les données chargées depuis les CSV, évitant ainsi de les relire à chaque lancement — ce qui représente un gain de temps significatif sur des fichiers de plusieurs milliers de lignes.

Git / GitHub est utilisé pour la gestion de versions et la collaboration au sein du groupe, conformément aux exigences du projet.

pytest est le framework utilisé pour les tests unitaires, en complément de la bibliothèque standard `unittest`.

Chapitre 3

Réalisation

3.1 Organisation du travail

[À COMPLÉTER — répartition des tâches entre les membres du groupe]

Le développement a été organisé autour du dépôt GitHub commun. *[À COMPLÉTER — décrire le workflow : branches, fréquence des commits, revues de code, outils de communication (Discord, etc.)]*

L'analyse du dépôt Git montre *[À COMPLÉTER — nombre de commits, contributions par auteur]*.

3.2 Développement

Plusieurs choix d'implémentation méritent d'être soulignés.

Le **pattern Dispatcher** est au cœur de notre architecture de chargement. Chaque famille de loaders dispose d'une méthode statique `load_all_*` qui reçoit le nom du sport et délègue au loader spécifique. Par exemple, `PlayerLoader.load_all_player("Tennis", dossier)` appelle successivement `TennisMenPlayerLoader` et `TennisWomenPlayerLoader` et fusionne leurs résultats. Ce pattern rend l'ajout d'un nouveau sport très simple : il suffit de créer un nouveau loader et d'ajouter un cas dans le dispatcher, sans toucher au reste du code.

L'héritage pour les matchs permet de traiter tous les matchs de manière uniforme dans la classe `Sport`, tout en conservant les attributs spécifiques à chaque discipline. La méthode `calculer_classement()` de `Sport` parcourt tous les matchs (quelle que soit leur sous-classe) et applique la même règle de points (3 points pour une victoire, 1 point par équipe pour un nul).

La persistance par pickle a été mise en place pour améliorer l'expérience utilisateur. Le chargement initial des données depuis les CSV peut prendre plusieurs secondes sur les gros jeux de données (football, basketball). Une fois sérialisés, les objets `Sport` sont rechargés en une fraction de seconde.

La recherche par sous-chaîne (`nom.lower() in nom_complet().lower()`) permet une recherche partielle et insensible à la casse. En cas de résultats multiples, l'application affiche la liste et invite l'utilisateur à choisir, gérant ainsi les cas d'homonymes.

[À COMPLÉTER — décrire d'autres parties importantes du développement spécifiques à votre groupe]

3.3 Difficultés rencontrées

La gestion des données manquantes a constitué notre première difficulté. Les fichiers CSV contiennent de nombreuses valeurs manquantes (NaN) pour des champs comme la date de naissance, la taille ou certaines statistiques de match. Nous avons dû systématiquement utiliser `pd.isna()` pour détecter ces valeurs et leur substituer `None` lors de la construction des objets Python, afin d'éviter des erreurs à l'exécution.

La gestion du répertoire de travail a posé un problème récurrent. Notre application utilise des chemins relatifs (`data/`, `objets/`) pour accéder aux fichiers. Selon l'environnement de lancement (bouton Run de VS Code, terminal, cloud Onyxia), le répertoire courant n'était pas toujours celui du projet, ce qui rendait les fichiers introuvables et produisait silencieusement des dictionnaires de joueurs vides. Nous avons résolu ce problème en ajoutant `os.chdir(os.path.dirname(os.path.abspath(__file__)))` au début de `__main__.py`, forçant ainsi le répertoire courant à être celui contenant le script.

L'hétérogénéité des formats CSV a nécessité un effort de parsing spécifique pour chaque sport. Les noms de colonnes, les types de données, les formats de dates et les structures de saisons diffèrent d'un jeu de données à l'autre. La création d'un loader dédié par sport nous a permis d'isoler cette complexité.

La compatibilité des fichiers pickle a également posé problème. Lorsque la définition d'une classe Python est modifiée après la création d'un fichier pickle, la désérialisation peut produire des objets avec des attributs manquants. La solution a été de supprimer les fichiers pickle et de recharger les données depuis les CSV.

[À COMPLÉTER — ajouter d'autres difficultés rencontrées par votre groupe]

Chapitre 4

Tests et validation

4.1 Stratégie de test

Nous avons adopté une approche de tests unitaires avec `pytest` et `unittest`. Chaque module important dispose de son propre fichier de test, organisé dans le répertoire `test/` qui reproduit la structure de `src/`.

Fichier de test	Module couvert
<code>test/Common/test_utils.py</code>	Utilitaires communs
<code>test/Model/test_Match.py</code>	Classe <code>Match</code> et sous-classes
<code>test/Model/test_Player.py</code>	Classe <code>Player</code>
<code>test/Model/test_Team.py</code>	Classe <code>Team</code>
<code>test/Parsers/test_BadmintonMatchLoader.py</code>	Loader matchs badminton
<code>test/Parsers/test_BadmintonPlayerLoader.py</code>	Loader joueurs badminton
<code>test/Parsers/test_FootballPlayerLoader.py</code>	Loader joueurs football
<code>test/Parsers/test_VolleyMatchLoader.py</code>	Loader matchs volleyball
<code>test/Parsers/test_VolleyballPlayerLoader.py</code>	Loader joueurs volleyball

[À COMPLÉTER — décrire plus précisément la stratégie et les données de test utilisées]

4.2 Résultats des tests

[À COMPLÉTER — résultats de `pytest test/ -v`]

[À COMPLÉTER — insérer ici la figure des résultats de tests]

4.3 Limites de l'application

L'interface CLI uniquement : L'application fonctionne exclusivement en mode terminal. Une interface graphique ou web (par exemple avec Streamlit) rendrait l'outil plus accessible à des utilisateurs non techniques.

Des données figées : Les loaders sont codés pour des fichiers nommés précisément (ex. `atp_matches_2024.csv`). L'application ne se met pas à jour automatiquement.

La sérialisation pickle fragile : Toute modification d'une classe Python après la création d'un fichier pickle peut rendre ce dernier incompatible. Une solution plus robuste serait d'utiliser SQLite ou JSON, avec un système de versionnage.

La classe PlayerSearch non implémentée : Le fichier `src/Analysis/PlayerSearch.py` contient une classe vide. Toute la logique de recherche est concentrée dans `__main__.py`, ce qui contrevient au principe de séparation des responsabilités.

Le fichier __main__.py trop long : Avec plus de 1 200 lignes, ce fichier mêle logique d'affichage, navigation dans les menus et calculs. Une refactorisation en modules distincts (Vue / Contrôleur) améliorerait la lisibilité et la maintenabilité.

La couverture de tests partielle : Le fichier `__main__.py` n'est pas couvert par des tests unitaires, car il implique des interactions utilisateur difficiles à simuler automatiquement.

[À COMPLÉTER — ajouter d'autres limites identifiées]

Chapitre 5

Conclusion

[À COMPLÉTER — bilan personnel et collectif]

Ce projet nous a permis de développer une application Python complète de bout en bout, de la lecture des données brutes jusqu'à leur visualisation graphique. Nous avons couvert cinq sports différents avec des fonctionnalités variées, en respectant les contraintes imposées (POO, Git, tests unitaires, interface CLI).

Sur le plan technique, nous avons acquis une maîtrise concrète de pandas pour la manipulation de données, de matplotlib pour la visualisation, et de Git pour le travail collaboratif. La conception en couches (Parsers / Model / Analysis / Interface) nous a appris l'importance de la séparation des responsabilités pour produire un code maintenable et extensible.

Si nous avions disposé de plus de temps, nous aurions souhaité développer une interface web avec Streamlit, intégrer une base de données SQLite pour remplacer les fichiers pickle, et améliorer significativement la couverture de tests.

[À COMPLÉTER — mot de fin]